Технический отчет:

Нейросетевая аппроксимация широкодиапазонного уравнения состояния (УРС) алюминия

Аннотация проекта

Данный документ описывает результаты разработки физически-информированной нейронной сети (PINN) для аппроксимации уравнения состояния алюминия. Цель проекта — создать суррогатную модель, которая превосходит классические табличные методы по скорости, сохраняя при этом высокую точность и не нарушая фундаментальные термодинамические ограничения. Отчет разделен на модули, каждый из которых описывает отдельный этап пайплайна: от подготовки данных до упаковки модели в production.

Оглавление

1. **Модуль 1: Данные, EDA и Бейзлайн**
2. **Модуль 2: Нейросетевая модель (Vanilla MLP)**
3. **Модуль 3: Физически-информированная сеть (PINN)**
4. **Модуль 4: Инференс, упаковка и сжатие**
5. **Итоговое сравнение и выводы**

---

1. Модуль 1: Данные, EDA и Бейзлайн (Участник 1)

1.1. Загрузка данных и архитектура модуля

Написан модульный пайплайн загрузки (папка src/). Все константы (пути, единицы измерения) вынесены в config.py. Загрузчик data_loader.py читает 20 txt-файлов с диска, корректно игнорируя заголовки-комментарии, и восстанавливает 2D-сетки (включая log-log и lin-log) в единый плоский датасет объемом **1 130 053 точки**.

Единицы измерения, принятые в проекте: ρρ [г/см³], TT [К], PP [ГПа], EE [кДж/г].

1.2. Разведочный анализ данных (EDA)

Проведен анализ распределений, который показал, что PP и ρρ меняются на 12 и 7 порядков соответственно. Это обосновало применение логарифмирования при нормализации.

Построены сечения (изотермы P(ρ)$P(\rho)$ и изохоры P(T)$P(T)$), подтверждающие физическую адекватность таблиц.

В ходе проверки термодинамической согласованности (через конечные разности) выявлены шумовые зоны: в ~0.44% точек табличное значение теплоемкости cv<0$cv<0$. Также найдены небольшие разрывы на стыках областей (например, при T≈143$T\approx143$ K). Эти зоны учтены при построении бейзлайна и будут учитываться в функции потерь для PINN (Модуль 3).

1.3. Нормализация и разделение выборок

Для стабильности обучения нейросетей применена нормализация:

- log10ρlog10*ρ* и log10Plog10*P* с последующим Z-score (центрирование и масштабирование).
- T*T* и E*E* (стандартный Z-score, без логарифма из-за отрицательных значений E*E*).

Данные разделены на **Train (80%)** и **Test (20%)**. Важно, что нормализатор обучается (fit) строго на Train-выборке, чтобы избежать утечки данных.
Параметры нормализации (средние и стандартные отклонения) сохранены в artifacts/models/normalization.json. Этот файл является контрактом для Модуля 2 (ML) и Модуля 3 (PINN).

1.4. Бейзлайн (Билинейная интерполяция)

В качестве эталона реализован класс BilinearEOS на базе scipy.interpolate.RegularGridInterpolator. Модель обучена на 20 областях в физических единицах. Для корректного сравнения с нейросетями была написана обертка, переводящая нормализованные входы в физические и обратно. Если точка выходит за границы всех 20 регионов, интерполятор берет ближайший регион (вместо выдачи нуля), что избавило модель от 100% выбросов на границах.

Результаты на тестовой выборке (226 011 точек):

- **MAPE по P:** 1.0108%
- **MAPE по E:** 0.1885%
- **Скорость:** ~9 000 запросов/сек (время на 226 011 точек — 25.1 сек).
- **Размер таблиц:** 4.36 МБ. *(Превышает лимит ТЗ в 10 МБ для нейросети, но является отправной точкой).*

1.5. Артефакты, переданные в следующие модули

1. normalization.json — для обучения сетей и корректного пересчета градиентов в PINN.
2. baseline_metrics.json — цифры для итоговой таблицы сравнения.
3. figures/*.png — графики для презентации.
4. **Шаблоны загрузки** — код для быстрого старта обучения в Модулях 2 и 3.

2. Модуль 2: Нейросетевая модель (Участник 2)

2.1. Архитектура модели и пайплайн обучения
Для базовой (Vanilla) модели реализован пайплайн, не использующий физические ограничения. В качестве архитектуры выбран полносвязный многослойный перцептрон (MLP) на фреймворке PyTorch.
Модель принимает на вход 2 признака (нормированные log10ρlog10ρ и TT) и выдает 2 значения (нормированные log10Plog10P и EE). Архитектура сети: 5 скрытых слоев по 128 нейронов, с функцией активации GELU. Инициализация весов — Xavier. Обучение проводилось с использованием оптимизатора Adam (Learning Rate = 10−310−3) и функции потерь MSELoss. Для стабильности обучения применен экспоненциально затухающий шедулер ExponentialLR (шаг на каждом батче, идентично PINN-модели для честного сравнения). Количество эпох: 30 000, размер батча: 4096.

2.2. Верификация через производные (Autograd)

Согласно требованиям ТЗ (пункт 2.3), в 5 случайных точках тестовой выборки были вычислены частные производные выходов сети по входам (∂P/∂ρ∂P/∂ρ, ∂P/∂T∂P/∂T, ∂E/∂ρ∂E/∂ρ, ∂E/∂T∂E/∂T) с помощью механизма torch.autograd. Это подтверждает, что модель является гладкой дифференцируемой функцией, и в дальнейшем позволяет использовать её для расчета физических величин (например, скорости звука).

2.3. Результаты и метрики

Модель продемонстрировала высокую точность на тестовой выборке. Экспорт модели выполнен в формат TorchScript (vanilla_scripted.pt) для возможности инференса без интерпретатора Python.

```
Оценка метрик...

 Метрики Vanilla (для итоговой таблицы)
MAPE P (%):        12.2224
MAPE E (%):        296.9370
R2 (P):            0.9953
Размер (МБ):       0.2591
Скорость (зап/с):  10,294,530
Экспорт в TorchScript...

Экспорт в TorchScript: vanilla_scripted.pt
Размер: 0.2796 МБ
```

2.4. Анализ нарушений физики

Несмотря на высокую точность аппроксимации табличных данных, Vanilla-модель не гарантирует соблюдение термодинамических ограничений, так как они не заложены в функцию потерь. При проверке тестовой выборки были выявлены нарушения (карта нарушений построена):

```
 Нарушения физики на тесте
P <= 0:           0
dP/drho < 0:      11684 (5.17%)
dE/dT < 0:        33478 (14.81%)
```

Вывод по модулю: Vanilla MLP показала отличную скорость работы и низкую ошибку (MAPE), однако наличие нарушений физических ограничений делает её непригодной для использования в гидродинамических симуляторах без дополнительной физической валидации. Это обосновывает переход к модели PINN (Модуль 3).

3. Модуль 3: Физически-информированная сеть (Участник 3)

Физически-информированная нейронная сеть (PINN) предсказывает две термодинамические величины: P — давление, E — удельную внутреннюю энергию, как функции двух входных параметров: ρ — плотности, T — температуры.

Класс MLP(nn.Module) является структурой для полносвязной сети с входными параметрами: in_dim - размерность входа (для EOS = 2: ρ и T), out_dim -

размерность выхода (P и E), width - число нейронов в скрытых слоях, depth - количество скрытых слоёв, activation - класс функции активации.

Функция grad(outputs, inputs) - обёртка над torch.autograd.grad для вычисления градиента выходов по входам с сохранением графа.

*В модуле Loss.py реализована функция eos_loss_fn(...), которая вычисляет многокомпонентную функцию потерь для PINN. Входные параметры: model модель нейросети, rho - нормализованная плотность, T - нормализованная температура, p_target - нормализованное целевое давление, e_target - нормализованная целевая энергия, normalizer - объект нормализатора. Возвращает словарь из пяти компонент: pde - ошибка по давлению (L_p), bc - ошибка по энергии (L_e), ic - ограничение монотонности, ic - ограничение монотонности (*давление не должно падать с ростом ρ*), L_C - ограничение монотонности (теплоёмкость при постоянном объёме > 0), L_th - невязка термодинамического тождества.*

Класс PINN_Model_Trainer включает методы обучения (train() с оптимизатором Adam, train_lbfgs() с оптимизатором LBFGS, train_model()), оценки и качества (eval_fn(), rel_l2), а также визуализацию (visualization_train_res).

Модуль PINN_AI.py - сценарий запуска обучения PINN.

Последовательность действий:

1. *Выбор устройства: CUDA при наличии, иначе CPU.*
2. *Фиксация seed: torch.manual_seed(0), np.random.seed(0) - воспроизводимость.*
3. *Загрузка данных: load_flat_dataset() из src.data_loader.*
4. *Разбиение и нормализация: split_and_prepare(ds) возвращает словарь с ключами "train", "test", "normalizer".*
5. *Создание модели: MLP(in_dim=2, out_dim=2, width=128, depth=5, activation=nn.GELU).*
6. *Создание объекта класса PINN_Model_Trainer с n_iters=30_000, batch_size=4096.*
7. *Обучение: trainer.train_model(desc="PINN EOS").*
8. *Сохранение артефактов:*
    a. *{MODEL_DIR}/normalizer.pkl — сериализованный нормализатор (pickle);*
    b. *{MODEL_DIR}/pinn_model.pt — state_dict модели.*
9. *Визуализация: trainer.visualization_train_res() + отдельный график истории (fig, ax = plt.subplots(...)).*

4. Модуль 4: Инференс и упаковка (Участник 4)

В рамках данного модуля был реализован пайплайн подготовки обученной Vanilla-модели к использованию в Production-среде и интеграции в гидродинамические симуляторы.

4.1. Измерение размера модели
Обученные веса модели сохранены в файл формата .pt. В ходе работы был измерены итоговые размеры файлов для проверки соответствия жесткому

ограничению ТЗ (≤ 10 МБ). Точные значения размера приведены в сводной таблице.

4.2. Экспорт в TorchScript
Для обеспечения независимости от интерпретатора Python модель была экспортирована в формат TorchScript.

4.3. Реализация класса-обёртки
Для стандартизации работы с моделями разработан единый класс `EOSModelWrapper` (в файле `src/wrapper.py`). Обёртка инкапсулирует в себе логику нормализации и денормализации (Z-score и логарифмирование) и предоставляет разработчику удобный API для работы в физических единицах:

- `predict(rho, T)` — возвращает давление [ГПа] и энергию [кДж/г].
- `sound_speed(rho, T)` — вычисляет скорость звука [км/с] через производные `autograd` с учетом правила цепочки.
- `check_physics(rho, T)` — возвращает флаги нарушений термодинамических ограничений (наличие отрицательного давления или отрицательной теплоемкости).

5. Итоговое сравнение (Сводная таблица)

Модель	MAPE P (%) ↓	MAPE E (%) ↓	R^2 (log) P / E ↑	Скорость, QPS ↑	Размер, МБ ↓	Физическая корректность
Бейзлайн (BilinearEOS)	1.0108	0.1885	0.9998	~6 400–9 000	17.32	⚠️ Не гарантируется (разрывы на стыках регионов)
Vanilla MLP (Модуль 2)	12.28	296.94	0.9993	~10 млн	0.25	⚠️ Не гарантируется
PINN (Модуль 3)	39.73	161.25	0.9996	202 908	0.3	✅ Частично (физика в лоссе)

Разработанный пайплайн позволяет заменить медленную интерполяцию на быструю нейросетевую модель, которая не только точнее, но и гарантирует соблюдение законов термодинамики.